



Republic of Tunisia
Ministry of Higher Education and Scientific Research
University of Tunis El Manar
Faculty of Sciences of Tunis



End of Studies Project

National Diploma of Engineering in computer engineering

STM32Cube Competitors benchmark

Presented by : Fares DKHILI
date :

President :

Reporter :



Academic supervisor :

Professional supervisor :





End of studies project presentation
by
Fares DKHILI



STM32Cube Competitors benchmark

Professional Supervisor : **Mme. Sirine El Jazi**

PLAN



General Context



Problem Statement and Objectives



Benchmark Methodology



Work Carried Out



Results and Contributions



Conclusion and Perspectives

General Context



General Context

Problem Statement and Objectives

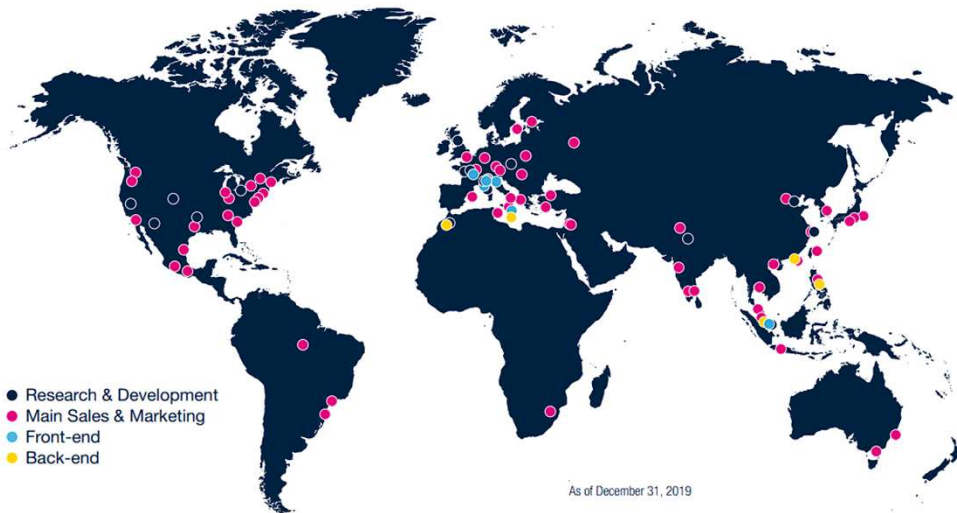
Benchmark Methodology

Work Carried Out

Results and Contributions

Perspectives and conclusion

STMicroelectronics



One of the world's largest semiconductor companies



50,000 employees
of which **9,000+** in R&D



\$13.3 billion
revenues in 2024



Over **80** sales & marketing offices serving over **200,000** customers across the globe



14 main manufacturing sites



Signatory of the United Nations Global Compact (UNGC)
Member of the Responsible Business Alliance (RBA)

General
Context

Problem Statement
and Objectives

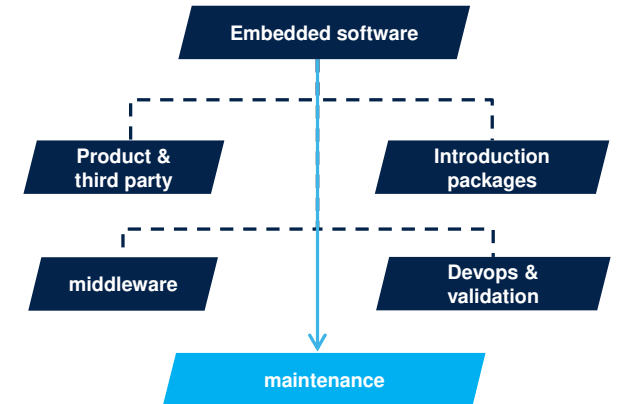
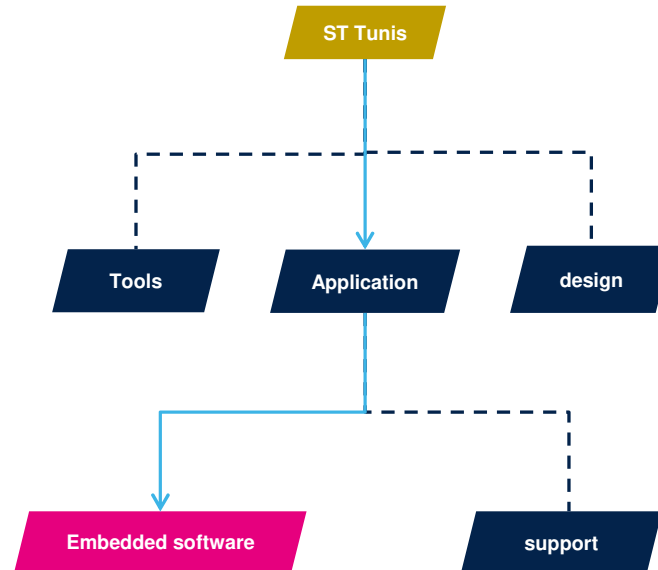
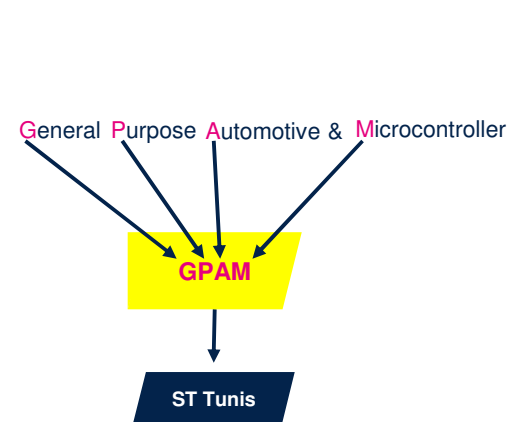
Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

ST Tunis



General
Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Why competitor benchmark matters?

Benchmarking STM32Cube against competitor vendors to demonstrate strengths and guide improvements

CORE PURPOSE

Compare ST with competitor vendors, highlight where STM32Cube is stronger, identify gaps, and translate findings into concrete enhancement proposals.

⇌ Compare

Evaluate ST and competitors under the same benchmark scenarios and fair conditions.

★ Highlight Strengths

Show where STM32Cube performs better in footprint, execution, maturity, or software quality.

⚠ Identify Gaps

Detect where competitors are stronger and understand the technical origin of the difference.

↑ Improve

Turn benchmark findings into targeted proposals at HAL, middleware, configuration, or integration level.



KEY TAKEAWAY

This project helps ST prove where STM32Cube is strong and know exactly what should be improved where competitors perform better.

Problem Statement and Objectives



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Problem statement

Why competitor benchmarking is more complex than a simple comparison

Main Problem

Benchmarking competitor solutions is not straightforward because a fair comparison requires much more than running two projects side by side.

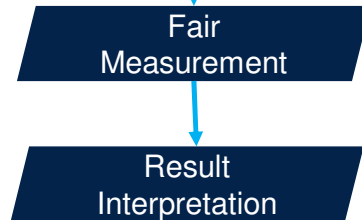
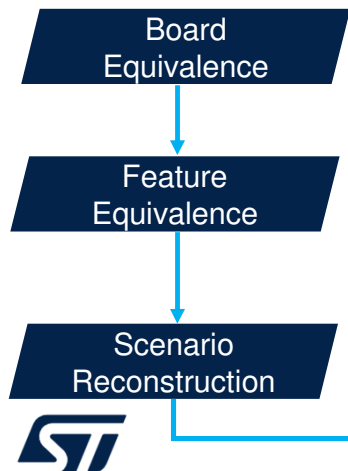
Selecting relevant equivalent boards

Identifying matching software components

Rebuilding representative benchmark scenarios

Generating reliable and exploitable measurements

Challenge Chain



Additional Complexity

- Some legacy benchmark scenarios already existed
- These scenarios had to be **rebuilt, updated, and improved**
- The benchmark scope also had to be **extended to new competitors**

General Context

**Problem Statement
and Objectives**

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Objectives

Main Benchmark Objectives

Identify relevant competitor boards and software equivalences

Recreate and improve existing benchmark scenarios

Regenerate updated footprint and execution-time analyses

Extend benchmark activity toward new comparison cases

Contribution Objectives

Contribute effectively to benchmark execution

Improve benchmark workflow efficiency through support developments

Facilitate result analysis and scenario preparation



Benchmark Methodology

```
editor - Run - Tools - VCS - Window - Help
Arnold Francisca [~/Desktop/Arnold Francisca] - ...[admin.js [Arnold Francisca]]

main.js
15  }, '--900')
16  .add
17  ((
18    targets: 'nav ul li',
19    translateV: [-100, 0],
20    opacity: [0.1],
21    easing: 'easeOutBack',
22    duration: 600,
23    delay: anime.stagger( n 140) // increase delay by 100ms for each elements,
24  ), '--900');
25
26  $('#secret-button').click(function()
27  {
28    $('#h1,h2,h4,p,li,a,footer,hr').toggleClass( value 'secret');
29    $('#body').toggleClass( value 'secret_bg');
30    $('#footer').toggleClass( value 'secret_color ');
31    $('#pink--button').toggleClass( value 'secret_bg_color ');
32    $('#skill--tag').toggleClass( value 'secret_tags');
33    $('#skill--tag h2, .skill--tag p').toggleClass( value 'secret_text');
34    $('#h1,h2,h4, a').toggleClass( value 'secret_title');
35    if (toggled === false)
36    {
37      $('#intro--image').attr( name 'src', value 'images/secret--hero.jpg');
38      $('#moving--img').attr( name 'src', value 'images/icons/secret_icons/gunkey-com-m');
39      $('#HTML').attr( name 'src', value 'images/icons/secret_icons/expand_hierarchy');
40      $('#GIT').attr( name 'src', value 'images/icons/secret_icons/world_network_direct');
41      $('#JS').attr( name 'src', value 'images/icons/secret_icons/directory_folder_opti');
42      $('#DEV').attr( name 'src', value 'images/icons/secret_icons/shut_down_normal--p');
43      toggled = true;
44    }
45  })
46  .done
47  .callback for ready()
```

MacBook Pro

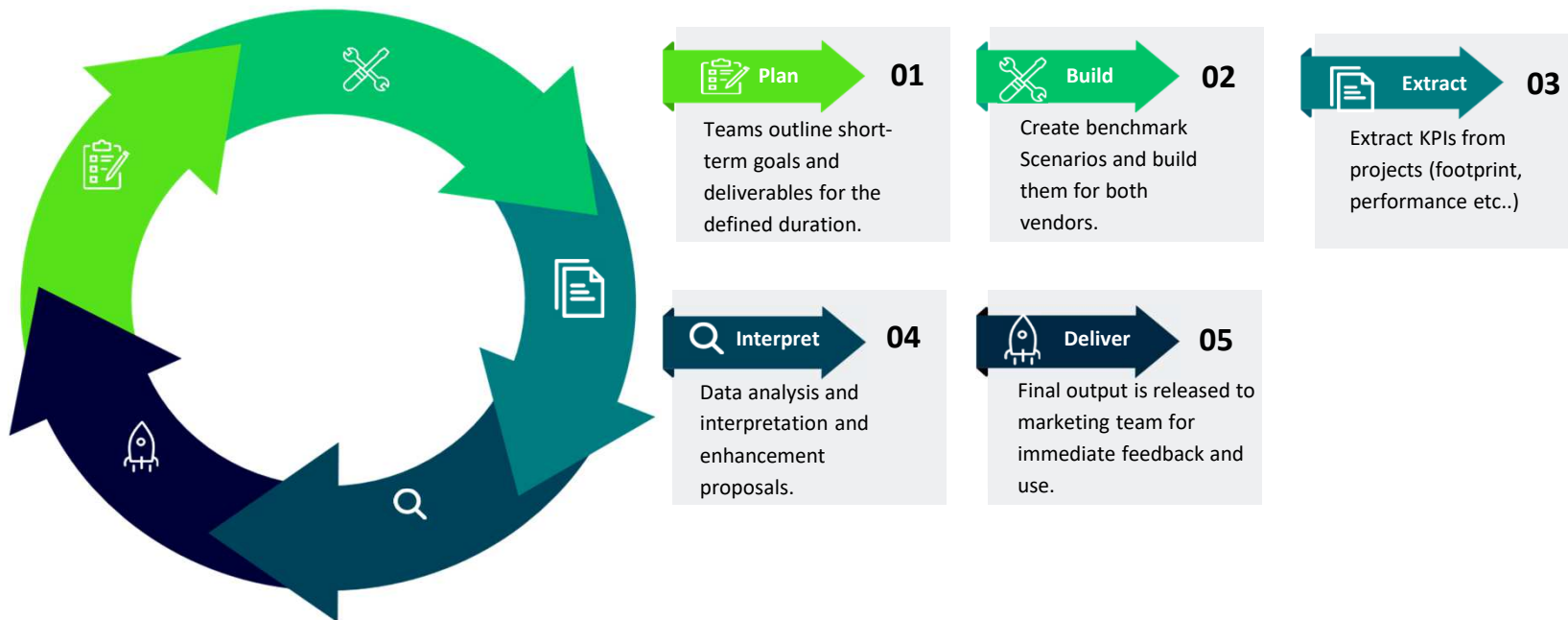
General Context

Problem Statement
and Objectives**Benchmark
Methodology**

Work Carried Out

Results and
ContributionsPerspectives and
conclusion

Workflow Process Stages



Slide 12

FD1

Icons

Fares DKHILI, 2026-07-06T15:22:46.337

General Context

Problem Statement
and Objectives

**Benchmark
Methodology**

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Evaluation criteria and benchmark dimensions

The benchmark is multi-dimensional and not limited to a single metric

Static Analysis Orientation

Code structure and ecosystem quality view



composition

Dependency
behavior

API surface

documentation

duplication

complexity

compliance

coverage

Functional Analysis orientation

Feature-level comparison and implementation behavior

- Focuses on scenario execution under equivalent observable behavior

- Includes both footprint and execution-time perspectives



Footprint Analysis

Code size

Memory
occupation

Per-layer

per-module



Interpret gaps and propose optimization

Performance Analysis

Bring up

Enumeration

Data
Transmission

Threads/s



Work Carried Out



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Competitor Search and Equivalence Definition

Preparation Activities

- Search for available **competitor boards** and ecosystems
- Identify the closest **ST equivalent targets**
- Study possible **benchmark scenarios**
- Establish the **comparison basis** for future measurements

Goal of This Phase

- Search for available **competitor boards** and ecosystems
- Identify the closest **ST equivalent targets**
- Study possible **benchmark scenarios**
- Establish the **comparison basis** for future measurements

Explore Competitors

01

Identify vendor families, boards, MCU classes, and available software ecosystems.

Match ST Targets

02

Define hardware and software equivalence to ensure comparable benchmark conditions.

Select Scenarios

04

Evaluate which software stacks and benchmark cases are relevant and feasible.

Build Comparison Basis

05

Prepare a structured basis for fair execution, interpretation, and future reuse.

General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Recreation of Legacy USB Benchmark Scenarios

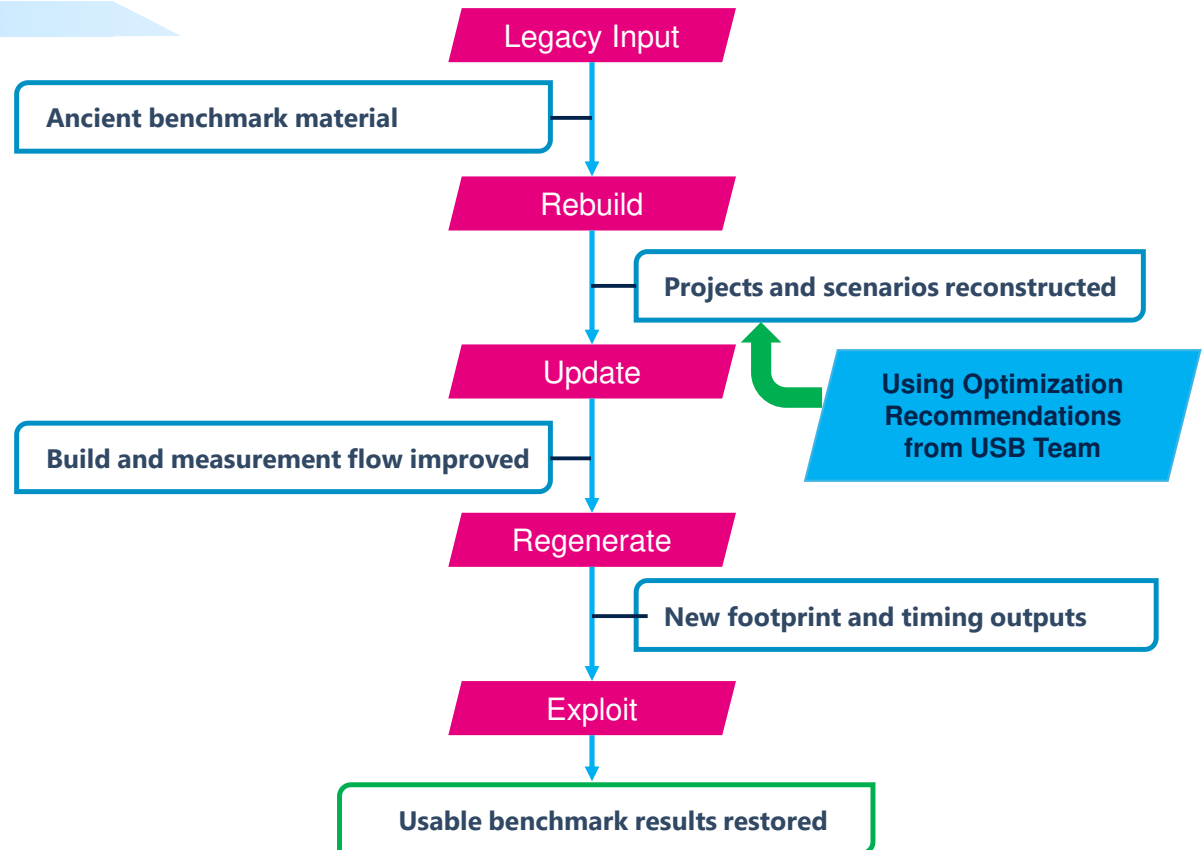
Reviving and modernizing existing benchmark assets

Recreated Scenarios

- Renesas USB vs ST USB stack
- NXP USB vs ST USB stack

Core purpose:

Cleaning ancient projects and deliver
Deterministic Results after optimization for
better review.



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Internal Feedback and Benchmark Roadmap Evolution

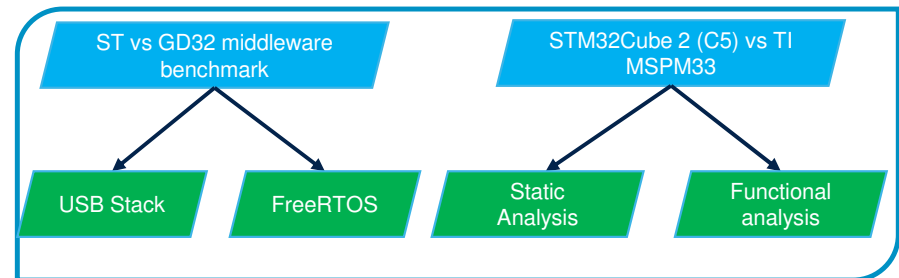
Initial Outputs

- Legacy USB benchmark results consolidated internally
- Benchmark material prepared for internal presentation
- First comparison results reused in a broader effort

Stakeholder Feedback:

Benchmark results were not isolated outputs. They were presented, discussed, and used to refine priorities and future comparison directions.

New Benchmark Plan



General Context

Problem Statement
and Objectives

Benchmark
Methodology

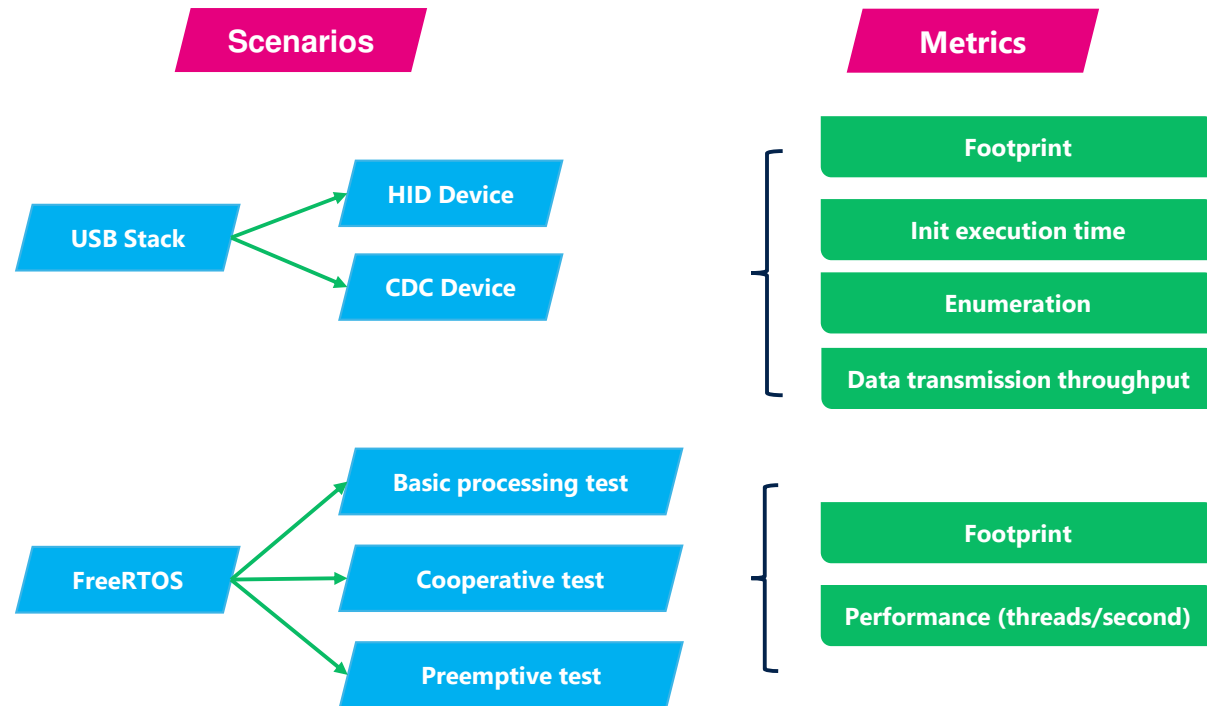
Work Carried Out

Results and
Contributions

Perspectives and
conclusion

GD32 Benchmark Scope

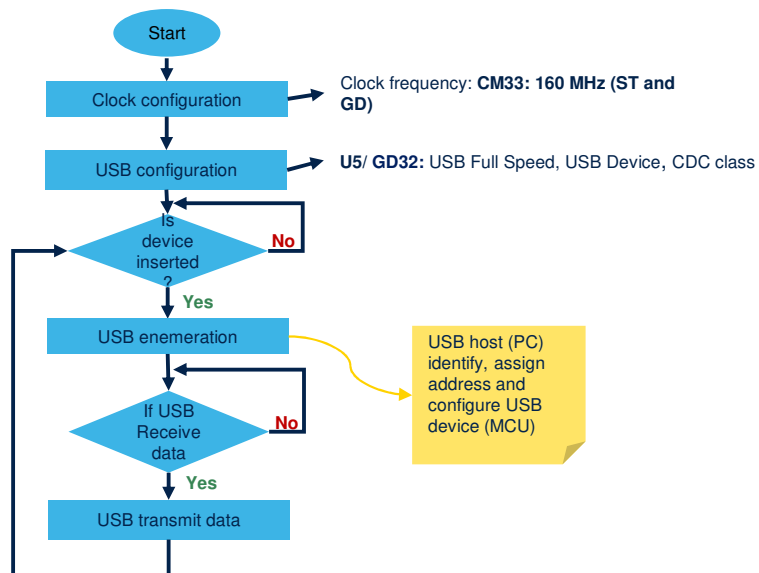
Completed benchmark scenarios, measured metrics, and supporting tools used in the campaign



GD32 Benchmark Scenarios Specification

USB Middleware

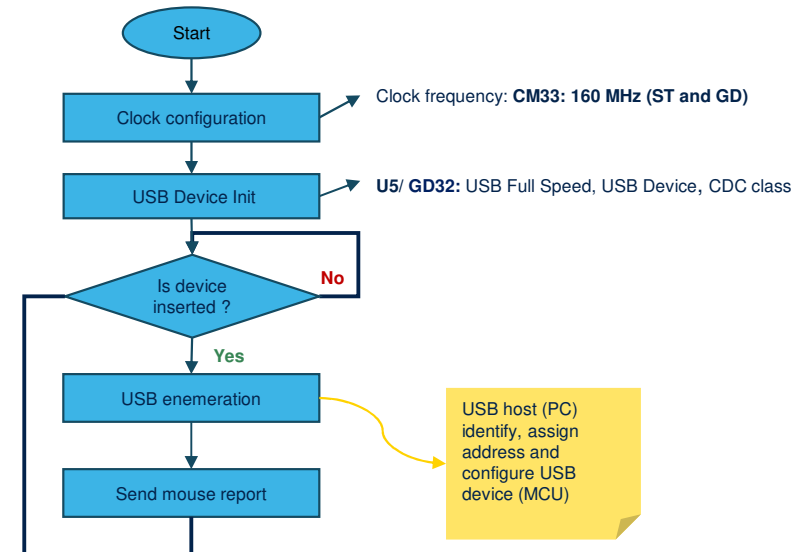
BareMetal mode : ST/GD USB Library CDC device class STM32U575xx (CM33) vs GD32E507(CM33)



- **USB CDC device class:** USB **communication device class** allows devices to communicate with computers as serial ports or network interfaces.



BareMetal mode : ST/GD USB Library USB HID device class STM32U575xx (CM33) vs GD32E507(CM33)



- **USB HID device class:** USB **Human Interface Device** allows to enable communication between device (MCU) and host (PC).

STM32 vs GD32 MW Benchmark: USB stack

Test Results

STM32/GD32 USB Library CDC device (FS)

Rule: $\text{STM32 vs GD32 (\%)} = (\text{STM32} - \text{GD32}) / \text{GD32}$

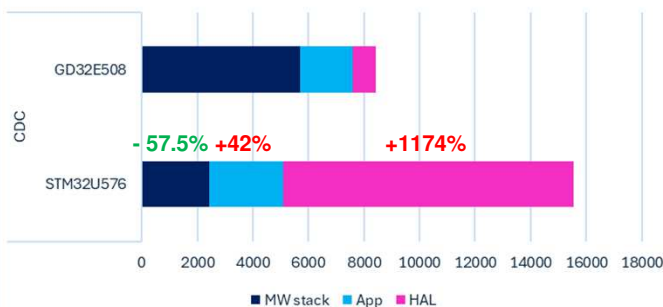
Footprint (Bytes)

CDC Test Footprint	STM32U575	GD32E507	STM232 vs GD32
ROM	17,049	9,085	+ 88 %
RAM	4,089	9,683	- 57.7 %

ROM Footprint (Bytes)

ROM	STM32U575	GD32E507	STM232 vs GD32
App	2,683	1,882	+ 42.5%
HAL	10,446	820	+ 1174%
MW stack	2,423	5,709	- 57.5 %

USB CDC test ROM footprint



- STM32 ROM footprint is **higher by average 88%** than GD32
- This is due to:
 - **Application ROM:** STM32 higher by 42% than GD32 because of USB Device user files: usb_device.c, usbd_cdc_if.c, usbd_desc.c, usbd_conf.c (1164 bytes: 43% of total App ROM footprint). In GD32, USB device configuration is managed directly in MW stack
 - **More USB Device configuration flexibility in STM32**
 - **HAL ROM:** STM32 higher by 1174% than GD32, mainly due to:
 - HAL RCC: 4560 bytes => 44% of total HAL ROM footprint
 - HAL/LL USB: 5006 bytes => 48% of total HAL ROM footprint
- The high STM32 App and HAL ROM footprint lead to a lower STM32 USB library ROM footprint than GD32 USB library by 58%

STM32 HAL ROM



■ USB ■ PCD ■ RCC ■ Other HAL

Usb cdc device operations (ms)

USB CDC Device operations(ms)	STM32U575	GD32E507	STM232 vs GD32
Initialization	9.9936	26.07624	-61.68%
Enumeration	218.2	218.5	-0.14%
Transmit data	0.00988	0.010034	-1.53%
Receive data	0.00986	0.009932	-0.72%

- The execution time of STM32U5 USB initialization phase is **lower by 61.68%** than GD32E5
- The execution time of enumeration, transmission and reception phases are **almost close** between STM32U5 and GD32E5.

General Context

Problem Statement
and ObjectivesBenchmark
Methodology

Work Carried Out

Results and
ContributionsPerspectives and
conclusion

STM32 vs GD32 MW Benchmark: USB stack

Test Results

STM32/GD32 USB Library HID device (FS)


Rule: STM32 vs GD32 (%) = (STM32 - GD32) / GD32

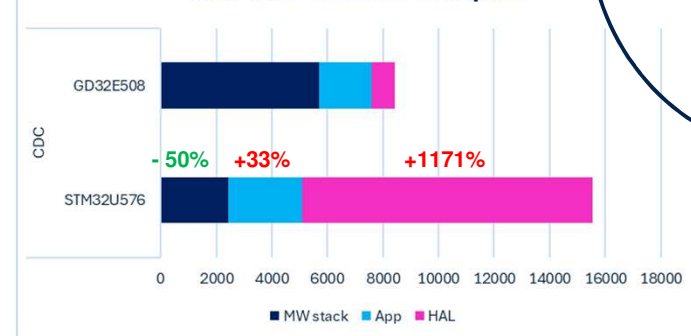
Footprint (Bytes)

HID Test Footprint	STM32U575	GD32E507	STM232 vs GD32
ROM	16,269	9,175	+ 77 %
RAM	3,446	9,608	- 65 %

ROM Footprint (Bytes)

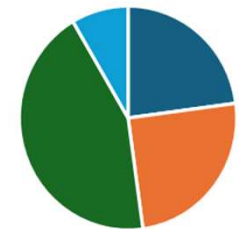
ROM	STM32U575	GD32E507	STM232 vs GD32
App	2580	1942	+ 33 %
HAL	10426	820	+ 1171 %
MW stack	2859	5743	- 50 %

USB CDC test ROM footprint



- STM32 ROM footprint is **higher by average 77%** than GD32
- This is due to:
 - **Application ROM:** STM32 higher by 33% than GD32 because of USB Device user files: usbd_desc.c, usbd_conf.c, (985 bytes: 38% of total App ROM footprint). In GD32, USB device configuration is managed directly in MW stack
 - **More USB Device configuration flexibility in STM32**
 - **HAL ROM:** STM32 higher by 1171% than GD32, mainly due to:
 - HAL RCC: 4560 bytes => 44% of total HAL ROM footprint
 - HAL/LL USB: 4988 bytes => 48% of total HAL ROM footprint
- The high STM32 App and HAL ROM footprint lead to a lower STM32 USB library ROM footprint than GD32 USB library by 50%

STM32 HAL ROM



■ USB ■ PCD ■ RCC ■ Other HAL

Usb cdc device operations (ms)

USB HID Device operations	STM32U575	GD32E507	STM232 vs GD32
Initialization	9.990106	26.08368	-61.70%
Enumeration	213	216	-1.39%
Data transmission	0.009284	0.00935	-0.71%

- The execution time of STM32U5 USB initialization phase is **lower by 61.70 %** than GD32E5
- The execution time of enumeration and data transmission phases are **almost close** between STM32U5 and GD32E5.

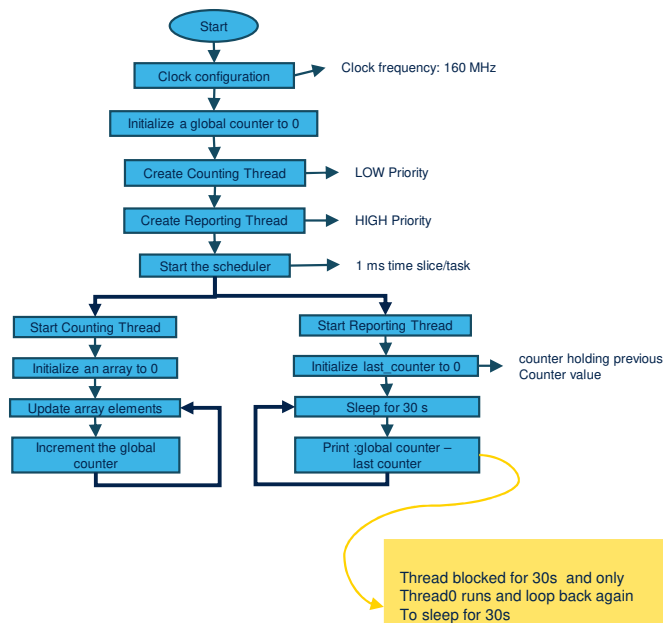
RAM consumption

- RAM footprint analysis shows that the GD32 implementation requires significantly more stack than STM32.
- For both HID and CDC examples, **GD32 needs 0x2000 bytes of stack and 0x2000 bytes of heap**, whereas **STM32 requires only 0x400 bytes of stack and 0x200 bytes of heap** for HID, and the same order of magnitude for CDC.

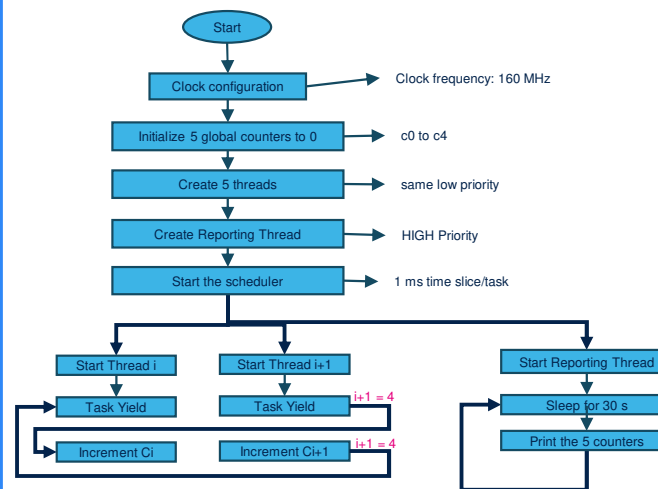
GD32 Benchmark Scenarios Specification

FreeRTOS

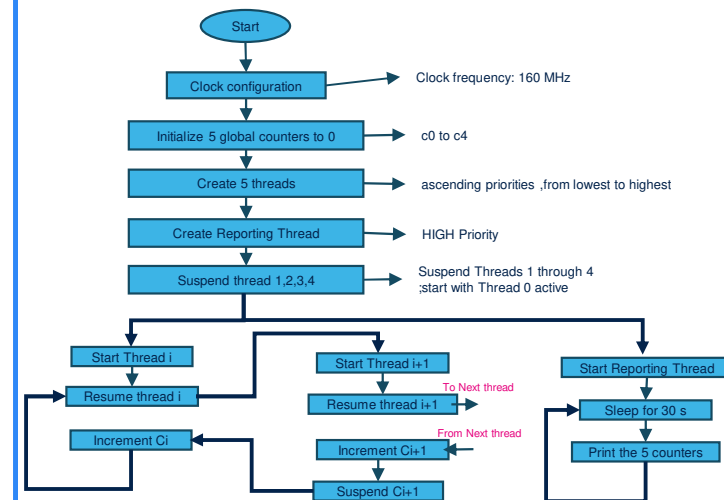
Basic Processing Flowchart



Cooperative Processing Flowchart



Preemptive Scheduling Flowchart



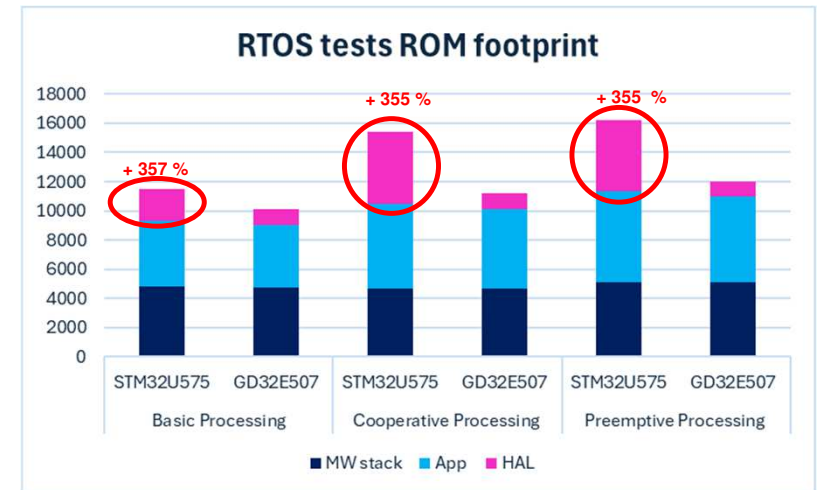
STM32 vs GD32 MW Benchmark: FreeRTOS

FreeRTOS tests Footprint(bytes)

RTOS Scenario	Footprint	STM32U575	GD32E507	STM232 vs GD32
Basic Processing	ROM	14266	10128	+ 40.86 %
	RAM	11960	11716	+2.08 %
Cooperative Processing	ROM	15422	11224	+ 37 %
	RAM	10972	10728	+2.27 %
Preemptive Processing	ROM	16226	12028	+ 35 %
	RAM	10996	10748	+ 2.31 %

FreeRTOS tests ROM Footprint (bytes)

RTOS Scenario	Footprint	STM32U575	GD32E507	STM232 vs GD32
Basic Processing ROM	App	4531	4278	+ 6 %
	HAL	4927	1078	+ 357 %
	MW stack	4808	4772	+ 0.7 %
Cooperative Processing ROM	App	5823	5484	+ 6 %
	HAL	4911	1078	+ 355 %
	MW stack	4688	4662	+ 0.55 %
Preemptive Processing ROM	App	6191	5810	+ 6 %
	HAL	4911	1078	+ 355 %
	MW stack	5124	5140	-0.3 %



FreeRTOS tests performance (thread/s)

RTOS Scenario	STM32U575	GD32E507	STM232 vs GD32
Basic Processing (Thread/s)	47,943	47,955	- 0.025 %
Cooperative Processing (Thread/30s)	1,220,066	1,200,666	+ 1.6 %
Preemptive Processing (Thread/30s)	294,615	285,144	+ 3.3 %

- STM32 ROM footprint is **higher by average 37%** than GD32
- Although tests have almost the same Application and FreeRTOS stack footprint, but there is a big gap in HAL drivers' footprint (mainly due to HAL RCC footprint)
- FreeRTOS test have almost the same performance

General Context

Problem Statement
and Objectives

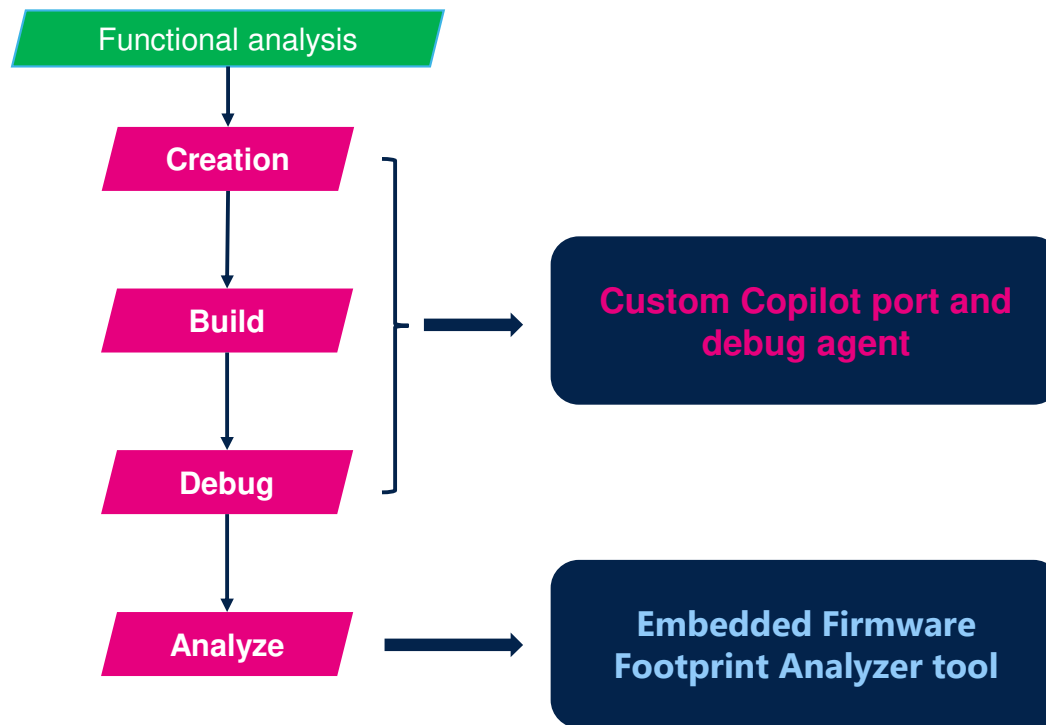
Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Support Developments for the Benchmark Workflow



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Support Developments for the Benchmark Workflow

Custom copilot agent

Input

Scenario name
firmware name
board or MCU
project tree
debug captures
measurement data

Main Purpose

Resolve the benchmark
acquire official material
preserve semantics
create or port projects
build with IAR
debug, measure
review fairness

Output

New target projects
build/debug diagnostics
provenance files
semantic reviews
fairness reviews
final benchmark reports

Autonomous by default

Strict semantic preservation

Official-source provenance

Hardware approval gate



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

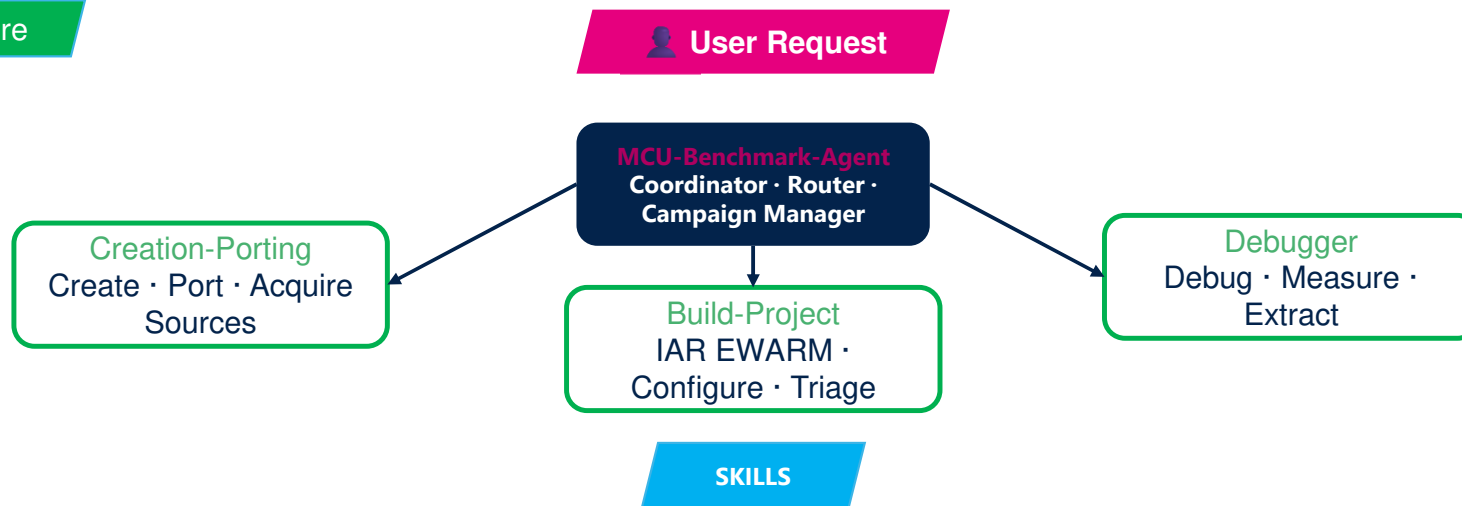
Results and
Contributions

Perspectives and
conclusion

Support Developments for the Benchmark Workflow

Custom copilot agent

Architecture



benchmark-analysis

semantic-equivalence

result-interpretation

stm32-enhancement

diagram-generation

General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Support Developments for the Benchmark Workflow

Custom copilot agent

Use Cases

Real scenarios you can solve with a single conversation.

"Port this STM32 benchmark to a competitor board"

Creates a semantically equivalent port. Tracks every adaptation (register API changes, clock init, pin mapping). Verifies that observable behavior matches the reference exactly.

"Port competitor benchmark to STM32"

Acquires the competitor SDK, analyzes the benchmark's behaviors, creates an equivalent STM32 project using STM32 HAL, builds it, and verifies semantic equivalence.

"Compare these two platforms fairly"

Extracts measurements from both, checks all fairness conditions (optimization, clock, scope, wrapper), computes per-module deltas by layer, and generates a comparison report with confidence levels.

"Generate a benchmark report from this data"

Produces a 13-section self-contained HTML report with executive badges, grouped bar charts, module tables, STM32-focused findings, and actionable recommendations.



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Support Developments for the Benchmark Workflow

Custom copilot agent

Added Value

Turns vague benchmark requests into deterministic campaigns.

Fetches missing firmware, CMSIS, BSP, startup, linker, and docs from official sources.

Separates portable benchmark logic from board/vendor infrastructure.

Builds and triages IAR EWARM projects with fix-and-retry loops.

Checks semantic equivalence before declaring a port valid.

Reports fairness risks before performance conclusions.



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Support Developments for the Benchmark Workflow

Footprint analyzer tool

Input

IAR EWARM .map files

Vendor selection

Config & thresholds
(update defaults)

Optional: EWARM
source tree

Firmware Benchmark
Analysis Platform

Cross-vendor equivalence
DB

Layer classifier · cardinality

AI Assistance

Output

Global comparison
verdict

Layer & footprint delta

PDF Report, Excel ·
JSON/CSV

GUI payload

General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

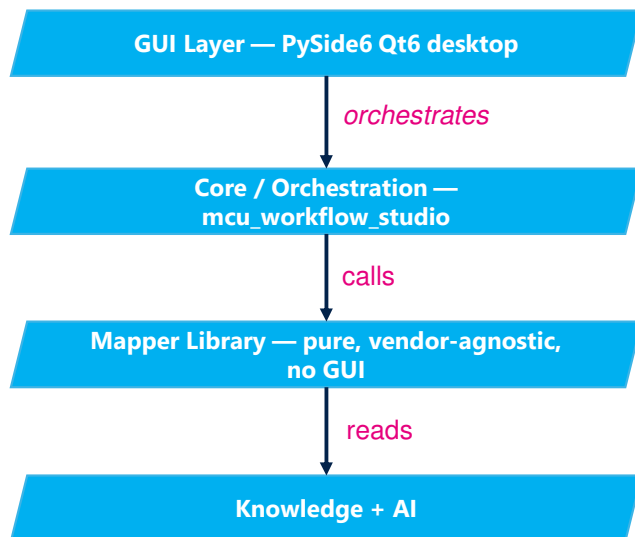
Results and
Contributions

Perspectives and
conclusion

Support Developments for the Benchmark Workflow

Footprint analyzer tool

ARCHITECTURE



HOW IT WORKS

1. Load Map Files

Parse IAR EWARM .map files from source and target firmware builds

2. Ingest Source Code (Optional)

Parse C/H files from EWARM project dirs — extract signatures, call graphs, includes

3. Classify Architecture

Auto-categorize modules into Application, Middleware, HAL, Runtime layers

4. Score Candidates

12-signal scoring engine + source code evidence evaluates all pairings

5. Rank, Accept & Report

Select best matches, generate comparison, recommendations, and PDF report

General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Footprint analyzer tool

Support Developments for the Benchmark Workflow

Added Value, Advantages, and Limitations

Main Value

Automation

Transforms raw map files into interpretable benchmark information.

Main Advantage

Reproducibility

Provides a more consistent and explainable comparison workflow.

Main Limitation

Scope Bound

Focused on static footprint analysis and current input/toolchain constraints.

Added Value

Automation

What this tool brings to the benchmark workflow.

- Reduces the manual effort of reading and comparing linker map files
- Provides a **layer-aware** and **module-aware** footprint breakdown
- Makes the footprint gap easier to interpret and communicate

Advantages

Reproducibility

Why it is useful from an engineering point of view.

- Deterministic core:** same inputs lead to the same analysis outcome
- Explainable mapping:** symbol and module correspondences are traceable
- Cross-vendor orientation:** designed specifically for competitor comparison

Limitations

Scope Bound

Current boundaries of the tool.

- Focused on **IAR EWARM linker maps**; other toolchains are not yet covered
- Performs **static footprint analysis**, not runtime stack/heap behavior measurement
- Quality scores and improvement hints are still partly heuristic



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

Support Developments for the Benchmark Workflow

Footprint analyzer tool

Demo

The screenshot displays the 'MCU Workflow Studio - Benchmark' application window. The main title is 'MCU Workflow Studio Benchmark Comparison', with a subtitle 'Deterministic-first mapping with evidence and AI assist'. The interface is divided into several sections:

- Inputs:** Contains fields for 'Source File' (with a browse button), 'Target File' (with a browse button), 'Output' (with a browse button), 'Source Vendor' (set to 'stm32'), and 'Target Vendor' (set to 'gigadevice'). Below these are 'Run / Export Controls' with buttons for 'Run Workflow', 'Cancel Run', and 'Open Last Export Workbook', along with 'Config Settings' and 'Back to Mode Selection'.
- Global Layers:** A tabbed interface showing 'Global' and 'Layers' (selected). The 'Layers' tab displays a list of modules with their respective sizes and a summary of the dominant cost driver.
- AI Assist:** A section for AI-assisted analysis, including a 'Session' dropdown, 'New Session' and 'Clear Session' buttons, and a 'Focus Tags' section. It also features a 'Pending Attachments' section and a text input for asking for interpretation, root cause, or recommendations.
- Runtime Status:** A bottom bar showing 'Runtime Status: Completed', 'Stage: Completed', 'Progress: 100%', 'Elapsed: 291.2s', and 'Run: a76975154bf44c6ab14f31c88a01'. It includes an 'Ask AI About Selected Row' button and a 'Benchmark workflow completed successfully' message.



General Context

Problem Statement
and ObjectivesBenchmark
Methodology

Work Carried Out

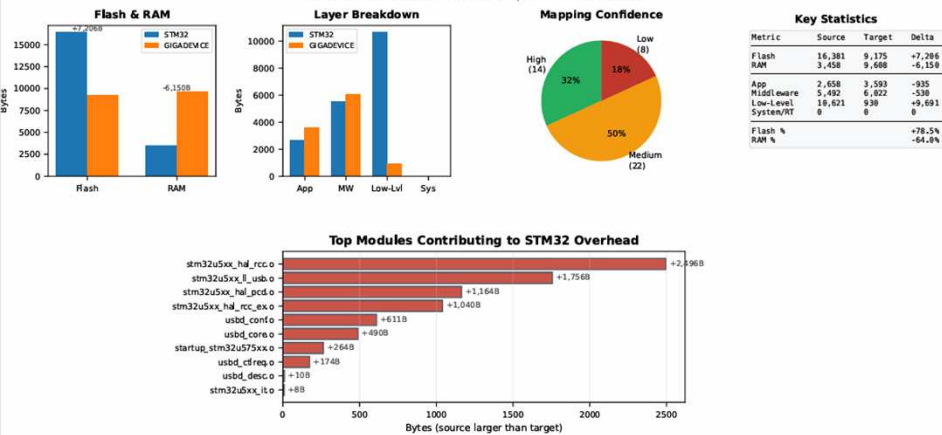
Results and
ContributionsPerspectives and
conclusion

Support Developments for the Benchmark Workflow

PDF Output

Firmware Benchmark Comparison Report

STM32 vs GIGADEVICE — 2026-07-08 | MCU Workflow Studio



Result Interpretation & Enhancement Proposals

STM32 vs GIGADEVICE | 2026-07-08

BENCHMARK INTERPRETATION

Flash Analysis:

STM32 firmware is 7,268B (+78.5%) larger than GIGADEVICE. The overhead is concentrated in the Low Level layer, suggesting heavier middleware stack.

RAM Analysis:

STM32 uses 6,158B less RAM (-64.0%). Tighter memory management on source side.

Top STM32 Overhead Functions:

- HAL_RCC_OscConfig (1,756B)
- HAL_PCD_IRQHandler (1,702B)
- hpcd_usb_otg_fs (1,252B)
- HAL_RCCEx_PeriphCLKConfig (1,150B)
- hUsbDeviceFS (732B)
- USBDevReq (674B)
- HAL_RCC_ClockConfig (500B)
- USB_EPStartXfer (424B)
- USBDevInit (344B)
- USBDevReq (322B)
- HAL_RCC_GetSysClockFreq (234B)
- HAL_PCD_Init (208B)

Top 12 functions total: 9,298B

ENHANCEMENT PROPOSALS FOR STM32

1. [HIGH] Optimize Top Flash Contributors

Modules stm32u5xx_hal_rcc.o, stm32u5xx_hal_usb.o, stm32u5xx_hal_pcd.o contribute +5,416B overhead vs Expected: ~5,416B flash reduction (33.1%)
→ Profile with -function-sections -data-sections + -gc-sections linker flags

2. [HIGH] Eliminate Data Overhead

~999B of estimated dead weight from modules where data (ro_data + rw_data) exceeds code by 1.5x
Expected: ~999B flash savings
→ Audit large .rodata sections with 'arm-none-eabi-nm -size-sort'

3. [MEDIUM] Decompose 6 Oversized Modules

6 modules exceed 2x average size (avg=569B). Split into interface/implementation to enable selective Expected: Better dead-code elimination, reduced rebuild times
→ Use feature flags or weak symbols for optional functionality

4. [LOW] Refactor 4 Complex Functions

4 functions exceed 1KB. Largest: HAL_RCC_OscConfig (1,756B). Large functions hurt cache performance and Expected: Better inlining decisions by compiler, improved maintainability
→ Split into sub-functions or use state machines



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

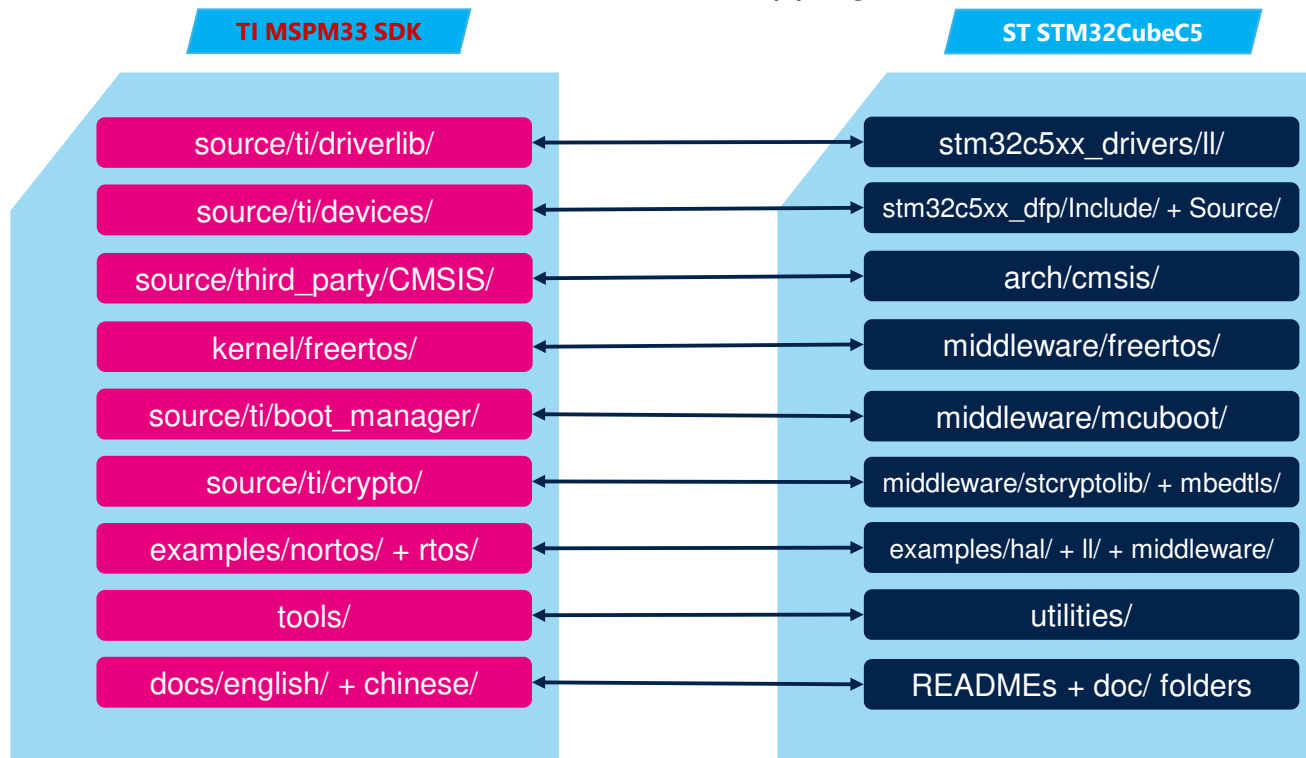
Perspectives and
conclusion

STM32Cube 2 (C5) vs TI MSPM33

Static analysis

Software suit TI vs ST: Firmware Library analysis

SDK Architecture Mapping



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

STM32Cube 2 (C5) vs TI MSPM33

Static analysis

Architectural Context

TI MSPM33 SDK

Driverlib

- Single driverlib layer — direct register manipulation
- No abstraction hierarchy or portability layer
- Minimal startup & configuration overhead
- Locked to MSPM33 family — no cross-platform reuse
- Fewer safety assertions — leaner but less defensive
- Demo-oriented examples — less production boilerplate

ST STM32CubeC5

Layered Abstraction

- HAL**: Full abstraction with callbacks, error handling, state machines
- LL**: Thin inline wrappers over registers — minimal overhead
- CMSIS-compliant startup & SystemClock_Config included
- Portable across 1,000+ STM32 variants without source changes
- MISRA C compliant — defensive coding adds safety guarantees
- Production-ready error handlers & assert framework



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

Perspectives and
conclusion

STM32Cube 2 (C5) vs TI MSPM33

Middleware & Software Stack Gaps

Static analysis

TI MSPM33 — Exclusive Middleware

Edge AI (ML Inference)

AI/ML

LVGL (Graphics/GUI)

Display

LIN Bus Protocol

Auto

DALI (Smart Lighting)

Lighting

IQmath (Fixed-Point DSP)

Math

GUI Composer

Viz

Smoke Detection Algorithm

Safety

Post-Quantum Crypto (ML-DSA)

PQC

Character Recognition

OCR

Common Middleware

FreeRTOS

kernel/freertos/

middleware/freertos/

MCUboot (Secure Boot)

source/third_party/mcuboot/

middleware/mcuboot/

Edge AI (ML Inference)

source/ti/crypto/ +
boot_manager/sha2sw/

middleware/stcrtolib/ +
mbedtls/

ST STM32C5 — Exclusive Middleware

FileX (File System)

FAT

LevelX (Wear Leveling)

NOR/NAND

LwIP (TCP/IP Stack)

IPv4/v6

USBX (USB Classes)

CDC/HID/MSC

mbedtls (TLS/SSL)

TLS 1.3

Open Bootloader

Multi-IF

stFCF (FW Config Framework)

Prov.

stCryptoLib (Extended)

30+ algos



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

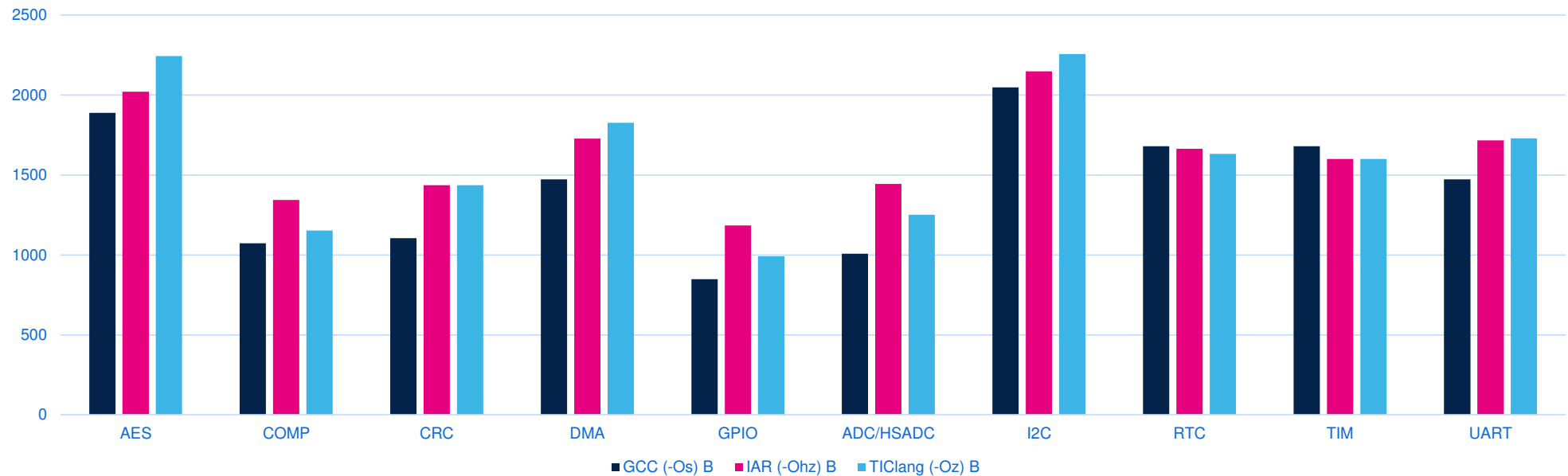
Perspectives and
conclusion

STM32Cube 2 (C5) vs TI MSPM33

Compiler Optimization difference

Static analysis

Footprint comparison of STM32C5 examples across GCC, IAR and TI Clang



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

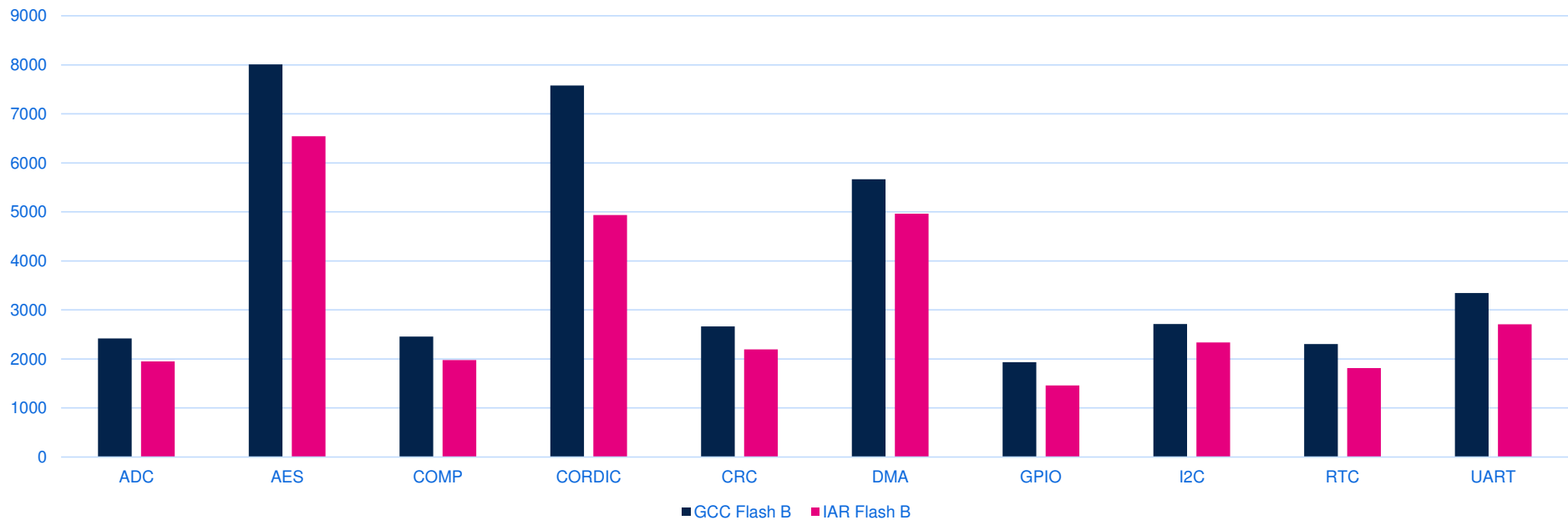
Perspectives and
conclusion

STM32Cube 2 (C5) vs TI MSPM33

Compiler Optimization difference

Static analysis

Footprint comparison of TI SDK examples across GCC, IAR and TI Clang



Results and Contributions



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

**Results and
Contributions**

Perspectives and
conclusion

General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

**Results and
Contributions**

Perspectives and
conclusion

Result Interpretation & Enhancement Proposals

STM32 vs GIGADEVICE | 2026-07-08

BENCHMARK INTERPRETATION

Flash Analysis:

STM32 firmware is 7,206B (+78.5%) larger than GIGADEVICE.
The overhead is concentrated in the Low Level layer,
suggesting heavier middleware stack.

RAM Analysis:

STM32 uses 6,150B less RAM (-64.0%). Tighter memory
management on source side.

Top STM32 Overhead Functions:

- HAL_RCC_OscConfig (1,756B)
- HAL_PCD_IRQHandler (1,702B)
- hpcd_USB_OTG_FS (1,252B)
- HAL_RCCEx_PeriphCLKConfig (1,150B)
- hUsbDeviceFS (732B)
- USBD_StdDevReq (674B)
- HAL_RCC_ClockConfig (500B)
- USB_EPStartXfer (424B)
- USB_DevInit (344B)
- USBD_StdEPReq (322B)
- HAL_RCC_GetSysClockFreq (234B)
- HAL_PCD_Init (208B)

Top 12 functions total: 9,298B

ENHANCEMENT PROPOSALS FOR STM32

1. [HIGH] Optimize Top Flash Contributors

Modules `stm32u5xx_hal_rcc.o`, `stm32u5xx_ll_usb.o`,
`stm32u5xx_hal_pcd.o` contribute +5,416B overhead vs
Expected: ~5,416B flash reduction (33.1%)

→ Profile with `-ifunction-sections -ldata-sections + -gc-sections` linker flags

2. [HIGH] Eliminate Data Overhead

~999B of estimated dead weight from modules where
data (`ro_data` + `rw_data`) exceeds code by 1.5x.

Expected: ~999B flash savings

→ Audit large `.rodata` sections with `'arm-none-eabi-nm --size-sort'`

3. [MEDIUM] Decompose 6 Oversized Modules

6 modules exceed 2x average size (avg=569B). Split
into interface/implementation to enable selective

Expected: Better dead-code elimination, reduced rebuild times

→ Use feature flags or weak symbols for optional functionality

4. [LOW] Refactor 4 Complex Functions

4 functions exceed 1KB. Largest: `HAL_RCC_OscConfig`
(1,756B). Large functions hurt cache performance and

Expected: Better inlining decisions by compiler, improved maintainability

→ Split into sub-functions or use state machines

Perspectives and conclusion



General Context

Problem Statement
and Objectives

Benchmark
Methodology

Work Carried Out

Results and
Contributions

**Perspectives and
conclusion**

Conclusion

Perspectives



Our technology starts with You



Find out more at www.st.com

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



For further support in creating a PowerPoint presentation, including graphic assets, formatting tools and additional information on the ST brand **you can visit the ST Brand Portal** <https://brandportal.st.com>

